

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM  
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG  
KHOA CÔNG NGHỆ THÔNG TIN**



**Lê Đức Huỳnh Đại - 52300013  
Nguyễn Khánh Duy - 52300019**

**BÁO CÁO GIỮA KỲ  
NHẬP MÔN  
XỬ LÝ NGÔN NGỮ TỰ NHIÊN**

**THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025**

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM  
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG  
KHOA CÔNG NGHỆ THÔNG TIN**



**Lê Đức Huỳnh Đại - 52300013  
Nguyễn Khánh Duy - 52300019**

**BÁO CÁO GIỮA KỲ  
NHẬP MÔN  
XỬ LÝ NGÔN NGỮ TỰ NHIÊN**

**Người hướng dẫn  
GV Trịnh Hùng Cường**

**THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025**

## LỜI CẢM ƠN

Đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến Thầy Trịnh Hùng Cường – Giảng viên khoa Công nghệ thông tin – Trường đại học Tôn Đức Thắng, đã hỗ trợ và giúp đỡ nhiệt tình trong quá trình thực hiện Dự án này.

Chúng em trân trọng cảm ơn Thầy Cô giảng viên Trường đại học Tôn Đức Thắng nói chung cũng như Thầy Cô giảng viên khoa Công nghệ thông tin nói riêng đã giảng dạy và truyền đạt nhiều kinh nghiệm quý trong suốt quá trình học tập tại trường.

Cuối cùng, xin cảm ơn gia đình, bạn bè đã luôn động viên và đồng hành trong quá trình học tập cũng như quá trình thực hiện Dự án này.

Mặc dù rất cẩn thận trong quá trình thực hiện đồ án cũng như viết báo cáo nhưng cũng không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý từ các Thầy/Cô để đồ án được hoàn thiện hơn.

Xin chân thành cảm ơn!

*TP. Hồ Chí Minh, ngày 01 tháng 11 năm 20..*

*Tác giả*

*(Ký tên và ghi rõ họ tên)*

**Lê Đức Huỳnh Đại**

**Nguyễn Khánh Duy**

## **CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG**

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của GV Trịnh Hùng Cường. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

**Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình.** Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

*TP. Hồ Chí Minh, ngày 01 tháng 11 năm 2025.*

*Tác giả*

*(Ký tên và ghi rõ họ tên)*

**Lê Đức Huỳnh Đại**

**Nguyễn Khánh Duy**

## TÓM TẮT

Giới thiệu về Xử lý Ảnh Số cung cấp kiến thức cơ bản về cách xử lý hình ảnh và video. Trong bài luận này, chúng em sẽ tìm hiểu cách sử dụng thư viện OpenCV để thao tác với hình ảnh và cách trích xuất ký tự từ hình ảnh. Học cách sử dụng thư viện OpenCV và các thư viện cơ bản trong Python để nhận diện biển báo giao thông và đánh dấu chúng theo yêu cầu của nhiệm vụ. Bằng cách áp dụng những kiến thức hữu ích mà tôi đã học trong khóa học, chúng ta có thể sử dụng phương pháp phù hợp để xử lý hình ảnh và video số.

## MỤC LỤC

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>DANH MỤC HÌNH VẼ .....</b>                                         | <b>vi</b> |
| <b>CHƯƠNG 1. PHƯƠNG PHÁP GIẢI QUYẾT BÀI TOÁN .....</b>                | <b>1</b>  |
| 1.1 Bài toán 1 .....                                                  | 1         |
| 1.1.1 Giới thiệu .....                                                | 1         |
| 1.1.2 Giai đoạn 1: Chuẩn bị và Khởi tạo .....                         | 1         |
| 1.1.3 Giai đoạn 2: Tiền xử lý hình ảnh (Pre-processing) .....         | 1         |
| 1.1.4 Giai đoạn 3: Tạo, làm sạch và tối ưu color mask .....           | 1         |
| 1.1.5 Giai đoạn 4: Tìm counter – Xác thực và phân loại biến báo ..... | 2         |
| 1.1.6 Giai đoạn 5: Loại bỏ trùng lặp .....                            | 3         |
| 1.1.7 Giai đoạn 6: Vẽ kết quả và xử lý video .....                    | 3         |
| 1.2 Giải thích chi tiết mã nguồn bài toán 1 .....                     | 3         |
| 1.3 Bài toán 2 .....                                                  | 16        |
| 1.3.1 Giới thiệu .....                                                | 16        |
| 1.3.2 Giai đoạn 1: Xử lý tổng thể (Global Processing): .....          | 16        |
| 1.3.3 Giai đoạn 2: Xử lý chuyên biệt theo ROI-based Processing: ..... | 16        |
| 1.4 Giải thích chi tiết mã nguồn Taks 2 .....                         | 17        |
| <b>CHƯƠNG 2. KẾT QUẢ CÁC BÀI TOÁN .....</b>                           | <b>26</b> |
| 2.1 Bài toán 1 .....                                                  | 26        |
| 2.1.1 Output 1 .....                                                  | 26        |
| 2.1.2 Output 2 .....                                                  | 27        |
| 2.1.3 Output 3 .....                                                  | 27        |
| 2.2 Bài toán 2 .....                                                  | 28        |

|                                 |           |
|---------------------------------|-----------|
| <b>CHƯƠNG 3. KẾT LUẬN .....</b> | <b>29</b> |
| 3.1 Kết luận .....              | 29        |
| 3.2 Hướng phát triển .....      | 29        |
| <b>TÀI LIỆU THAM KHẢO .....</b> | <b>30</b> |
| 3.3 Tiếng Việt .....            | 30        |
| 3.4 Tiếng Anh .....             | 30        |

**DANH MỤC HÌNH VẼ**

|                                               |    |
|-----------------------------------------------|----|
| Hình 1 : Output 1 .....                       | 26 |
| Hình 2 : Output 2 .....                       | 27 |
| Hình 3 : Output 3 .....                       | 27 |
| Hình 4    Kết quả output.png bài toán 2 ..... | 28 |



# CHƯƠNG 1. PHƯƠNG PHÁP GIẢI QUYẾT BÀI TOÁN

## 1.1 Bài toán 1

### 1.1.1 Giới thiệu

Bài toán sử dụng phương pháp Computer Vision truyền thống để phát hiện và phân loại biển báo giao thông từ video. Hệ thống tiếp cận vấn đề theo hướng kết hợp màu sắc và hình dạng: đầu tiên phát hiện các vùng có màu đặc trưng (đỏ, xanh dương, vàng) trong không gian màu HSV, sau đó xác thực hình dạng học (hình tròn, tam giác, chữ nhật/vuông) để phân loại biển báo. Quy trình bao gồm tiền xử lý ảnh với CLAHE để cải thiện độ tương phản, tạo mask màu, tìm contour, xác thực hình dạng và áp dụng Non-Maximum Suppression để loại bỏ phát hiện trùng lặp. Phương pháp này tối ưu về tốc độ xử lý và không cần huấn luyện mô hình học sâu.

### 1.1.2 Giai đoạn 1: Chuẩn bị và Khởi tạo

- Import các thư viện cần thiết: `'cv2'` (OpenCV), `'numpy'`, `'time'`
- Khởi tạo các đối tượng tái sử dụng để tối ưu tốc độ:
  - + CLAHE (Contrast Limited Adaptive Histogram Equalization) để điều chỉnh ánh sáng
  - + Kernel cho các phép toán hình thái học (morphology operations)
  - + Các mảng HSV range cho màu đỏ, xanh dương và vàng

### 1.1.3 Giai đoạn 2: Tiền xử lý hình ảnh (Pre-processing)

- Điều chỉnh ánh sáng: Chuyển frame sang không gian màu LAB, áp dụng CLAHE trên kênh độ sáng (L) để cải thiện độ tương phản, đặc biệt trong điều kiện ánh sáng yếu
- Chuyển đổi không gian màu: Chuyển từ BGR sang HSV để dễ dàng phát hiện màu sắc

### 1.1.4 Giai đoạn 3: Tạo, làm sạch và tối ưu color mask

- Tạo các mask nhị phân để phát hiện các màu đặc trưng của biển báo:

- Mask màu đỏ: Phát hiện biển báo cấm (hình tròn đỏ), do màu đỏ nằm ở hai đầu của vòng tròn HSV (0-10 và 160-180), cần tạo 2 mask riêng và kết hợp
- Mask màu xanh dương: Phát hiện biển báo chỉ dẫn (hình tròn hoặc chữ nhật/vuông)
- Mask màu vàng: Phát hiện biển cảnh báo (hình tam giác vàng với viền đỏ)
- Áp dụng Gaussian blur để giảm nhiễu trong các mask
- Thực hiện phép toán hình thái học (morphological closing) để đóng các lỗ nhỏ và làm liền các vùng bị ngắt quãng

#### ***1.1.5 Giai đoạn 4: Tìm counter – Xác thực và phân loại biển báo***

- Sử dụng `cv2.findContours()` để tìm các vùng liên thông trong từng mask
- Chỉ lấy các contour ngoài cùng (RETR\_EXTERNAL) với phương pháp xấp xỉ đơn giản (CHAIN\_APPROX\_SIMPLE)
- Biển báo hình tròn (đỏ hoặc xanh):
  - + Kiểm tra độ tròn bằng công thức circularity:  $4\pi \times \text{area} / \text{perimeter}^2$
  - + Kiểm tra diện tích hợp lý
  - + Sử dụng Hough Circles để xác nhận hình dạng tròn
- Biển báo hình chữ nhật/vuông (xanh dương)\*\*:
  - + Kiểm tra số đỉnh (phải có 4 đỉnh)
  - + Kiểm tra tỷ lệ khung hình (aspect ratio)
  - + Kiểm tra tỷ lệ điền đầy (fill ratio)
  - + Xác nhận màu xanh dương trong ROI
- Biển báo hình tam giác (vàng với viền đỏ)\*\*:
  - + Kiểm tra số đỉnh (phải có 3 đỉnh)
  - + Kiểm tra diện tích hợp lý
  - + Xác nhận tỷ lệ màu vàng và màu đỏ trong ROI

### 1.1.6 Giai đoạn 5: Loại bỏ trùng lặp

- Áp dụng thuật toán NMS để loại bỏ các bounding box trùng lặp
- Tính IoU (Intersection over Union) giữa các box
- Chỉ giữ lại box có điểm số cao nhất trong mỗi nhóm box trùng lặp

### 1.1.7 Giai đoạn 6: Vẽ kết quả và xử lý video

- Vẽ bounding box màu đỏ lên các biển báo đã phát hiện
- Vẽ MSSV lên từng frame
- Ghi frame đã xử lý vào video đầu ra
- Hiện thị tiến trình xử lý với phần trăm và FPS

## 1.2 Giải thích chi tiết mã nguồn bài toán 1

### Bước 1: Khởi tạo biến toàn cục

- **Mục tiêu:** Tạo một lần các đối tượng và mảng sẽ được sử dụng nhiều lần trong quá trình xử lý, tránh tạo lại mỗi frame để tối ưu tốc độ.

```
# Tạo CLAHE một lần để tái sử dụng (tối ưu tốc độ - không ảnh hưởng
độ chính xác)
_clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
# Tạo kernel cho morphology operations một lần để tái sử dụng
_morphology_kernel = np.ones((5, 5), np.uint8)
# Tạo các mảng HSV range một lần để tái sử dụng (tối ưu tốc độ)
_lower_red1 = np.array([0, 50, 50])
_upper_red1 = np.array([10, 255, 255])
_lower_red2 = np.array([160, 50, 50])
_upper_red2 = np.array([180, 255, 255])
_lower_blue = np.array([100, 100, 100])
_upper_blue = np.array([140, 255, 255])
_lower_yellow = np.array([15, 80, 120])
_upper_yellow = np.array([35, 255, 255])
```

**Bước 2: Tạo hàm Điều chỉnh ánh sáng của frame để cải thiện chất lượng hình ảnh. (adjust\_lighting(frame, clahe=None))**

- **Mục tiêu:** CLAHE giúp làm sáng vùng tối và điều chỉnh độ tương phản theo từng vùng nhỏ, cải thiện khả năng phát hiện biến báo trong điều kiện ánh sáng kém.
- **Quy trình:**
  1. Chuyển đổi từ BGR sang LAB color space
  2. Tách kênh độ sáng (L)
  3. Áp dụng CLAHE trên kênh L để tăng độ tương phản
  4. Merge lại các kênh và chuyển về BGR
- **Mã nguồn – giải thích chi tiết:**

```
# Hàm điều chỉnh ánh sáng của frame để cải thiện chất lượng hình ảnh
def adjust_lighting(frame, clahe=None):
    # Chuyển đổi sang không gian màu LAB để tách thành phần độ sáng
    lab = cv2.cvtColor(frame, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)

    # Áp dụng CLAHE (Contrast Limited Adaptive Histogram Equalization) để cải thiện độ tương phản
    # CLAHE giúp làm sáng vùng tối và điều chỉnh độ tương phản theo từng vùng nhỏ
    # Sử dụng CLAHE đã tạo sẵn để tránh tạo lại mỗi frame (tối ưu tốc độ)
    if clahe is None:
        clahe = _clahe
    l = clahe.apply(l)

    # Merge các kênh màu lại với nhau
    lab = cv2.merge((l, a, b))

    # Chuyển đổi ngược lại về không gian màu BGR để hiển thị
    adjusted_frame = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
    return adjusted_frame
```

**Bước 3: Tạo các mask nhị phân để phát hiện màu đỏ, xanh dương và vàng (get\_color\_masks(frame\_hsv))**

- **Mục tiêu:** Trả về 3 mask tương ứng với 3 màu.
- **Xử lý đặc biệt cho màu đỏ:**

- + Màu đỏ nằm ở hai đầu vòng tròn HSV (0-10° và 160-180°)
- + Tạo 2 mask riêng và kết hợp bằng phép OR (|)
- **Mã nguồn – giải thích chi tiết:**

```
# Hàm tạo các color mask để phát hiện màu đỏ, xanh dương và vàng
trong frame HSV
def get_color_masks(frame_hsv):
    # Tạo mask cho màu đỏ với khoảng giá trị rộng hơn để phát hiện
    tốt hơn
    # Màu đỏ nằm ở hai đầu của vòng tròn màu HSV (0-10 và 160-180)
    # Sử dụng các mảng đã tạo sẵn để tối ưu tốc độ

    # Tạo hai mask cho hai vùng màu đỏ và kết hợp chúng lại
    red_mask1 = cv2.inRange(frame_hsv, _lower_red1, _upper_red1)
    red_mask2 = cv2.inRange(frame_hsv, _lower_red2, _upper_red2)
    red_mask = red_mask1 | red_mask2

    # Tạo mask cho màu xanh dương với khoảng giá trị điều chỉnh để
    phát hiện tốt hơn
    blue_mask = cv2.inRange(frame_hsv, _lower_blue, _upper_blue)

    # Tạo mask cho màu vàng (dùng cho biển cảnh báo tam giác)
    yellow_mask = cv2.inRange(frame_hsv, _lower_yellow,
    _upper_yellow)

    return red_mask, blue_mask, yellow_mask
```

#### **Bước 4: Phát hiện hình tròn bằng thuật toán Hough Circles.**

- **Mục tiêu: Phát hiện hình tròn bằng thuật toán Hough Circles.**
- **Các bước:**
  1. Chuyển sang ảnh xám
  2. Áp dụng Gaussian blur để giảm nhiễu
  3. Gọi “cv2.HoughCircles()” với các tham số đã tinh chỉnh:
    - “dp=1.5”: Độ phân giải tích lũy
    - “minDist=35”: Khoảng cách tối thiểu giữa các tâm tròn
    - “param1=60”: Ngưỡng cao của Canny edge detector
    - “param2=25”: Ngưỡng tích lũy (accumulator threshold)
- **Mã nguồn – giải thích chi tiết:**

```
# Hàm phát hiện các hình tròn trong frame bằng thuật toán Hough
Circles
def detect_circles(frame, min_radius=15, max_radius=80):
    # Chuyển đổi frame sang ảnh xám để xử lý
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Áp dụng Gaussian blur để giảm nhiễu và cải thiện khả năng phát
    hiện hình tròn
    # Kernel size nhỏ hơn, độ lệch chuẩn cao hơn để phát hiện cạnh tốt
    hơn
    blurred = cv2.GaussianBlur(gray, (7, 7), 2.5)

    # Thực hiện phát hiện hình tròn bằng Hough Circle với các tham số
    đã được tinh chỉnh
    circles = cv2.HoughCircles(
        blurred,
        cv2.HOUGH_GRADIENT,
        dp=1.5,                # Điều chỉnh dp để kiểm soát độ chính
        xác phát hiện tốt hơn
        minDist=35,            # Khoảng cách tối thiểu giữa các tâm
        tròn (giảm để phát hiện các tròn gần nhau)
        param1=60,             # Ngưỡng cao của Canny edge detector
        (thấp hơn để nhạy hơn với cạnh)
        param2=25,             # Ngưỡng tích lũy (thấp hơn để phát hiện
        nhạy hơn)
        minRadius=min_radius,  # Bán kính tối thiểu của hình tròn
        maxRadius=max_radius   # Bán kính tối đa của hình tròn
    )

    if circles is not None:
        # Làm tròn và trả về các hình tròn đã phát hiện được
        return np.uint16(np.around(circles[0]))
    return None
```

## Bước 5: Kiểm tra contour

- Mục tiêu: Phân biệt các loại biển báo theo hình dạng, đặc điểm
- Tiêu chí kiểm tra
  1. Hình tròn
    - Diện tích: loại bỏ contour quá nhỏ ( $< 0.2$ ) hoặc quá lớn ( $> 10000$ )
    - Độ tròn (circularity): “ $4\pi \times \text{area} / \text{perimeter}^2$ ” phải  $\geq 0.7$
  2. Hình vuông/ chữ nhật xanh dương
    - Diện tích so với frame: 0.1% - 25%
    - Số đỉnh: phải có đúng 4 đỉnh (sử dụng “approxPolyDP”)

- Tỷ lệ khung hình (aspect ratio): 0.5 - 1.8
- Tỷ lệ điền đầy (fill ratio):  $\geq 0.4$
- Tỷ lệ pixel màu xanh trong ROI:  $\geq 30\%$

### 3. Tam giác viền đỏ

- Diện tích so với frame: 0.1% - 20%
- Số đỉnh: phải có đúng 3 đỉnh
- Tỷ lệ pixel màu vàng trong ROI:  $\geq 20\%$
- Tỷ lệ pixel màu đỏ trong ROI:  $\geq 3\%$  (viền)

## - Mã nguồn – giải thích chi tiết:

### 1. Hình tròn

```
# Hàm kiểm tra xem contour có phải là biến báo hình tròn hay không
def verify_sign(frame, contour):
    # Tính toán các thuộc tính hình dạng của contour
    area = cv2.contourArea(contour)
    perimeter = cv2.arcLength(contour, True)

    # Lọc theo diện tích - loại bỏ các contour quá nhỏ hoặc quá lớn
    if area < 0.2 or area > 10000: # Điều chỉnh các ngưỡng này theo
nhu cầu
        return False

    # Tính toán độ tròn (circularity) để kiểm tra xem có phải hình
tròn không
    # Công thức: 4*pi*area/perimeter^2, giá trị càng gần 1 càng tròn
    circularity = 4 * np.pi * area / (perimeter * perimeter)
    if circularity < 0.7: # Không đủ tròn (ngưỡng 0.7)
        return False

    return True
```

### 2. Hình vuông/ chữ nhật xanh dương

```
# Hàm kiểm tra xem contour có phải là biến báo hình chữ nhật/vuông
màu xanh dương hay không
def verify_blue_rectangle(frame, contour):
    # Lấy kích thước frame và tính diện tích frame
    h, w = frame.shape[:2]
    frame_area = float(w * h)
    area = cv2.contourArea(contour)
```

```

# Kiểm tra diện tích contour so với frame (phải trong khoảng hợp lý)
if area < 0.001 * frame_area or area > 0.25 * frame_area:
    return False

# Tính chu vi và xấp xỉ contour thành đa giác để kiểm tra số cạnh
peri = cv2.arcLength(contour, True)
approx = cv2.approxPolyDP(contour, 0.03 * peri, True)
# Hình chữ nhật/vuông phải có 4 đỉnh
if len(approx) != 4:
    return False

# Lấy bounding rectangle của contour
x, y, bw, bh = cv2.boundingRect(contour)
if bw == 0 or bh == 0:
    return False

# Kiểm tra tỷ lệ khung hình (aspect ratio) - phải trong khoảng hợp lý cho hình chữ nhật
aspect_ratio = bw / float(bh)
if aspect_ratio < 0.5 or aspect_ratio > 1.8:
    return False

# Kiểm tra tỷ lệ điền đầy (fill ratio) - contour phải chiếm đủ diện tích trong bounding box
fill_ratio = area / float(bw * bh)
if fill_ratio < 0.4:
    return False

# Lấy vùng quan tâm (ROI) và kiểm tra màu xanh dương trong vùng đó
roi = frame[y:y+bh, x:x+bw]
if roi.size == 0:
    return False

# Chuyển ROI sang HSV và tạo blue mask
hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
_, blue_roi, _ = get_color_masks(hsv_roi)
# Tính tỷ lệ pixel màu xanh trong ROI
blue_ratio = float(np.count_nonzero(blue_roi)) / float(blue_roi.size)
return blue_ratio >= 0.3

```

### 3. Hình tam giác viền đỏ



```

#Hàm kiểm tra xem contour có phải là biên cảnh báo hình tam giác vàng
với viền đỏ hay không
def verify_yellow_triangle(frame, contour):
    # Lấy kích thước frame và tính diện tích frame
    h, w = frame.shape[:2]
    frame_area = float(w * h)
    area = cv2.contourArea(contour)

    # Kiểm tra diện tích contour so với frame (phải trong khoảng hợp
    lý)
    if area < 0.001 * frame_area or area > 0.2 * frame_area:
        return False

    # Tính chu vi và xấp xỉ contour thành đa giác để kiểm tra số cạnh
    peri = cv2.arcLength(contour, True)
    approx = cv2.approxPolyDP(contour, 0.04 * peri, True)
    # Hình tam giác phải có 3 đỉnh
    if len(approx) != 3:
        return False

    # Lấy bounding rectangle của contour
    x, y, bw, bh = cv2.boundingRect(contour)
    if bw == 0 or bh == 0:
        return False

    # Lấy vùng quan tâm (ROI) và kiểm tra màu vàng và đỏ trong vùng đó
    roi = frame[y:y+bh, x:x+bw]
    if roi.size == 0:
        return False

    # Chuyển ROI sang HSV và tạo color mask
    hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
    red_roi, _, yellow_roi = get_color_masks(hsv_roi)
    # Tính tỷ lệ pixel màu vàng và màu đỏ trong ROI
    yellow_ratio = float(np.count_nonzero(yellow_roi)) /
float(yellow_roi.size)
    red_ratio = float(np.count_nonzero(red_roi)) / float(red_roi.size)
    # Phải có đủ màu vàng (phần trong) và màu đỏ (viền)
    return yellow_ratio >= 0.2 and red_ratio >= 0.03

```

### Bước 6: Loại bỏ các bounding box trùng lặp.

#### - Quy trình:

1. Sắp xếp các box theo điểm số giảm dần
2. Lặp qua từng box (từ điểm cao đến thấp):
  - Giữ lại box hiện tại
  - Tính IoU giữa box hiện tại với các box còn lại

- Loại bỏ các box có IoU > ngưỡng (0.4)

### 3. Trả về danh sách các box cần giữ lại

IoU (Intersection over Union): “diện tích giao nhau / diện tích hợp nhất”

#### - Mã nguồn – giải thích chi tiết

```
# Hàm thực hiện Non-Maximum Suppression (NMS) để Loại bỏ các hộp giới hạn (bounding box) trùng lặp
def non_max_suppression(boxes, scores, iou_thresh=0.4):
    # Nếu không có box nào thì trả về danh sách rỗng
    if len(boxes) == 0:
        return []

    # Chuyển đổi sang numpy array để xử lý
    boxes_np = np.array(boxes)
    scores_np = np.array(scores)

    # Tính toán tọa độ các góc của các box
    # Format: boxes = [(x, y, w, h), ...]
    x1 = boxes_np[:, 0]
    y1 = boxes_np[:, 1]
    x2 = boxes_np[:, 0] + boxes_np[:, 2]
    y2 = boxes_np[:, 1] + boxes_np[:, 3]

    # Tính diện tích của từng box
    areas = (x2 - x1 + 1) * (y2 - y1 + 1)

    # Sắp xếp các box theo điểm số giảm dần
    order = scores_np.argsort()[::-1]
    keep = []

    # Lặp qua các box theo thứ tự điểm số từ cao xuống thấp
    while order.size > 0:
        i = order[0] # Lấy box có điểm cao nhất
        keep.append(i) # Giữ lại box này

        # Tính toán vùng giao nhau (intersection) giữa box hiện tại
        # với các box còn lại
        xx1 = np.maximum(x1[i], x1[order[1:]])
        yy1 = np.maximum(y1[i], y1[order[1:]])
        xx2 = np.minimum(x2[i], x2[order[1:]])
        yy2 = np.minimum(y2[i], y2[order[1:]])
        w = np.maximum(0.0, xx2 - xx1 + 1)
        h = np.maximum(0.0, yy2 - yy1 + 1)
        inter = w * h
```

```

        # Tính IoU (Intersection over Union) - tỷ lệ giao nhau
        iou = inter / (areas[i] + areas[order[1:]] - inter)

        # Chỉ giữ lại các box có IoU thấp hơn ngưỡng (không trùng lặp
        nhiều)
        inds = np.where(iou <= iou_thresh)[0]
        order = order[inds + 1]

    return keep

```

### Bước 7: Xác định biển báo giao thông trong một frame

#### - Quy trình:

1. Điều chỉnh ánh sáng
2. Chuyển sang HSV và tạo color mask
3. Làm sạch mask bằng Gaussian blur và morphology
4. Tìm contour trong từng mask
5. Xác thực và phân loại từng contour:
  - Contour đỏ → kiểm tra hình tròn
  - Contour xanh → kiểm tra hình tròn hoặc chữ nhật/vuông
  - Contour vàng → kiểm tra hình tam giác
6. Áp dụng NMS để loại bỏ trùng lặp
7. Vẽ bounding box lên frame

#### - Mã nguồn – Giải thích chi tiết

```

# Hàm chính để phát hiện biển báo giao thông trong frame
def detect_traffic_signs(frame, clahe=None):
    # Điều chỉnh ánh sáng để cải thiện chất lượng hình ảnh
    # Truyền CLAHE đã tạo sẵn để tối ưu tốc độ
    adjusted_frame = adjust_lighting(frame, clahe)

    # Chuyển đổi sang không gian màu HSV để dễ dàng phát hiện màu sắc
    frame_hsv = cv2.cvtColor(adjusted_frame, cv2.COLOR_BGR2HSV)

    # Tạo các color mask cho màu đỏ, xanh dương và vàng
    red_mask, blue_mask, yellow_mask = get_color_masks(frame_hsv)

    # Áp dụng Gaussian blur để làm mịn và giảm nhiễu trong các mask
    red_mask = cv2.GaussianBlur(red_mask, (5, 5), 0)
    blue_mask = cv2.GaussianBlur(blue_mask, (5, 5), 0)
    yellow_mask = cv2.GaussianBlur(yellow_mask, (5, 5), 0)

    # Thực hiện các phép toán hình thái học để làm sạch mask (đóng các
    lỗ nhỏ)

```

```

# Sử dụng kernel đã tạo sẵn để tối ưu tốc độ
red_mask = cv2.morphologyEx(red_mask, cv2.MORPH_CLOSE,
_morphology_kernel)
blue_mask = cv2.morphologyEx(blue_mask, cv2.MORPH_CLOSE,
_morphology_kernel)
yellow_mask = cv2.morphologyEx(yellow_mask, cv2.MORPH_CLOSE,
_morphology_kernel)

# Tìm các contour trong các mask để phát hiện các vùng có màu
tương ứng
contours_red, _ = cv2.findContours(red_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
contours_blue, _ = cv2.findContours(blue_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
contours_yellow, _ = cv2.findContours(yellow_mask,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Danh sách lưu các biển báo đã phát hiện và điểm số tương ứng
detected_signs = []
detected_scores = []

# Xử lý các contour màu đỏ - thường là biển báo cấm (hình tròn)
for contour in contours_red:
    if verify_sign(frame, contour):
        x, y, w, h = cv2.boundingRect(contour)
        roi = frame[y:y+h, x:x+w]

        # Kiểm tra hình tròn bằng Hough Circles
        circles = detect_circles(roi)

        if circles is not None:
            detected_signs.append((x, y, w, h))
            detected_scores.append(float(cv2.contourArea(contour)))

# Xử lý các contour màu xanh dương - có thể là biển báo hình tròn
hoặc hình chữ nhật/vuông
for contour in contours_blue:
    # 2 nhánh: tròn (giữ nguyên) hoặc chữ nhật/vuông xanh
    x, y, w, h = cv2.boundingRect(contour)
    roi = frame[y:y+h, x:x+w]
    if roi.size == 0:
        continue
    # Kiểm tra hình tròn (tái sử dụng logic hiện có)
    if verify_sign(frame, contour):
        circles = detect_circles(roi)
        if circles is not None:

```

```

        detected_signs.append((x, y, w, h))
        detected_scores.append(float(cv2.contourArea(contour)))
        continue
    # Kiểm tra hình chữ nhật/vuông màu xanh
    if verify_blue_rectangle(frame, contour):
        detected_signs.append((x, y, w, h))
        detected_scores.append(float(cv2.contourArea(contour)))

    # Xử lý các contour màu vàng - thường là biển cảnh báo hình tam
    giác với viền đỏ
    for contour in contours_yellow:
        if verify_yellow_triangle(frame, contour):
            x, y, w, h = cv2.boundingRect(contour)
            detected_signs.append((x, y, w, h))
            detected_scores.append(float(cv2.contourArea(contour)))

    # Sử dụng NMS để loại bỏ các bounding box trùng lặp từ nhiều nhánh
    xử lý
    keep = non_max_suppression(detected_signs, detected_scores,
                               iou_thresh=0.4)

    # Vẽ các bounding box lên frame để đánh dấu biển báo đã phát hiện
    for i in keep:
        x, y, w, h = detected_signs[i]
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255), 2)

    return frame

```

**Bước 8: Xử lý toàn bộ video, phát hiện biển báo trong từng frame và ghi video đầu ra.**

– **Quy trình**

1. Mở video đầu vào và lấy thông tin (kích thước, FPS, tổng số frame)
2. Tạo VideoWriter để ghi video đầu ra
3. Vòng lặp xử lý từng frame:
  - Đọc frame
  - Phát hiện biển báo
  - Thêm Mã số sinh viên
  - Ghi frame
  - Cập nhật và hiển thị tiến trình với FPS
4. Giải phóng tài nguyên và hiển thị thống kê

– **Mã nguồn – Giải thích chi tiết**

```

# Hàm xử lý video: đọc video đầu vào, phát hiện biến báo trong từng
frame và ghi video đầu ra
def process_video(input_path, output_path):
    # Bắt đầu đo thời gian xử lý để tính FPS
    start_time = time.time()

    # Mở video đầu vào
    cap = cv2.VideoCapture(input_path)
    if not cap.isOpened():
        print("Error: Could not open video.")
        return

    # Lấy thông tin về video: chiều rộng, chiều cao và FPS
    frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = int(cap.get(cv2.CAP_PROP_FPS))
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    print(f"Starting video processing: {total_frames} frames")

    # Tạo VideoWriter để ghi video đầu ra với codec XVID
    fourcc = cv2.VideoWriter_fourcc(*'XVID')
    out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width,
frame_height))

    # Cài đặt text để hiển thị MSSV trên frame
    font = cv2.FONT_HERSHEY_SIMPLEX
    font_scale = 1
    color = (0, 255, 0) # Màu xanh lá
    thickness = 2
    line_type = cv2.LINE_AA # Anti-aliasing để text mượt hơn
    pos_id1 = (20, 40) # Vị trí MSSV thứ nhất (x, y)
    pos_id2 = (20, 80) # Vị trí MSSV thứ hai (x, y)

    # Đếm số frame đã xử lý để hiển thị tiến trình
    frame_count = 0

    # Đọc và xử lý từng frame trong video
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        # Phát hiện biến báo giao thông trong frame (truyền CLAHE đã
tạo sẵn để tối ưu)
        processed_frame = detect_traffic_signs(frame, _clahe)

```

```

        # Vẽ hai MSSV lên frame
        cv2.putText(processed_frame, '52300019', pos_id1, font,
font_scale, color, thickness, line_type)
        cv2.putText(processed_frame, '52300013', pos_id2, font,
font_scale, color, thickness, line_type)

    # Ghi frame đã xử lý vào video đầu ra
    out.write(processed_frame)

    # Cập nhật và hiển thị tiến trình với FPS
    frame_count += 1
    progress = (frame_count / total_frames) * 100

    # Tính FPS hiện tại (dựa trên thời gian xử lý từ đầu)
    current_time = time.time()
    elapsed_time = current_time - start_time
    current_fps = frame_count / elapsed_time if elapsed_time > 0
else 0

    # In tiến trình mỗi 1% hoặc mỗi frame nếu video nhỏ (dưới 100
frames)
    if frame_count % max(1, total_frames // 100) == 0 or
frame_count == total_frames:
        print(f"Progress: {progress:.1f}%
({frame_count}/{total_frames} frames) | FPS: {current_fps:.1f}",
end='\r')

    print() # Xuống dòng sau khi hoàn thành

    # Tính tổng thời gian xử lý
    total_time = time.time() - start_time
    avg_fps = frame_count / total_time if total_time > 0 else 0

    # Giải phóng tài nguyên
    cap.release()
    out.release()
    cv2.destroyAllWindows()
    print(f"Processing complete. Total time: {total_time:.2f}s | Avg
FPS: {avg_fps:.2f}")
    print(f"Output saved to: {output_path}")

# Chạy xử lý video
input_video_path = 'bài toán1.mp4'
output_video_path = 'bài toán1_output.avi'
process_video(input_video_path, output_video_path)

```

## 1.3 Bài toán 2

### 1.3.1 Giới thiệu

Để giải quyết bài toán phát hiện ký tự số trong ảnh input.png, một chiến lược xử lý ảnh "lai" (hybrid) đã được xây dựng. Cách tiếp cận này kết hợp một phương pháp xử lý tổng thể để phát hiện các ký tự rõ ràng, cùng với một loạt các quy trình xử lý chuyên biệt, được tinh chỉnh cho từng Vùng Quan Tâm (Region of Interest - ROI) cụ thể chứa nhiều nặng hoặc ký tự bị biến dạng.

### 1.3.2 Giai đoạn 1: Xử lý tổng thể (Global Processing):

- Mục tiêu: Nhanh chóng phát hiện và khoanh vùng phần lớn các ký tự số trong ảnh bằng một thuật toán chung.

- Hành động:

- + Ảnh đầu vào được chuyển sang ảnh thang độ xám.
- + Sử dụng Phân ngưỡng Thích ứng (Adaptive Thresholding) để tạo ảnh nhị phân. Kỹ thuật này hiệu quả hơn phân ngưỡng toàn cục vì nó tính toán ngưỡng cho từng vùng nhỏ của ảnh, giúp xử lý tốt hơn các thay đổi nhỏ về độ sáng và tương phản.
- + Tìm tất cả các đường viền (contours) trên ảnh nhị phân.
- + Áp dụng một bộ lọc ban đầu dựa trên diện tích để loại bỏ các đối tượng quá nhỏ hoặc quá lớn.
- + Sử dụng một hàm tùy chỉnh (merge\_boxes\_vertically) để hợp nhất các đường viền bị vỡ (ví dụ: số 8 bị tách thành hai nửa) thành một bounding box duy nhất.
- + Vẽ các bounding box đã xử lý lên ảnh kết quả.

### 1.3.3 Giai đoạn 2: Xử lý chuyên biệt theo ROI-based Processing:

- Mục tiêu: Giải quyết các trường hợp khó mà phương pháp tổng thể bỏ sót hoặc xử lý sai, đặc biệt là các vùng có nhiều dày đặc che khuất ký tự.

- Hành động:



- + Một danh sách các tọa độ ROI được xác định trước bằng tay, khoanh vùng chính xác các khu vực có vấn đề.
- + Đối với mỗi ROI, một bộ tham số xử lý riêng (kích thước kernel, ngưỡng nhị phân cố định, số lần lặp co/giãn) được áp dụng.
- + Một hàm chuyên dụng (detect\_and\_draw) được gọi để xử lý từng ROI với bộ tham số tùy chỉnh của nó. Quy trình bên trong hàm này bao gồm các phép biến đổi hình thái học, phân ngưỡng, tìm và hợp nhất đường viền tương tự như giai đoạn 1 nhưng được tinh chỉnh để tối ưu cho vùng ảnh nhỏ đó.
- + Các bounding box kết quả từ mỗi ROI được vẽ trực tiếp lên ảnh gốc, bổ sung cho kết quả của giai đoạn xử lý tổng thể.

## 1.4 Giải thích chi tiết mã nguồn Taks 2

### Bước 1: Khởi tạo và Tiền xử lý tổng thể

- Mục tiêu: Nạp thư viện, đọc ảnh và thực hiện bước phân ngưỡng đầu tiên trên toàn bộ ảnh để có cái nhìn tổng quan.

```
import cv2 as cv
import numpy as np

# Đọc ảnh gốc và chuyển sang grayscale
img = cv.imread("input.png")
grayImg = cv.imread("input.png", 0)

# Tạo ảnh nhị phân bằng adaptive thresholding
binary_img = cv.adaptiveThreshold(
    src=grayImg,
    maxValue=255,
    adaptiveMethod=cv.ADAPTIVE_THRESH_GAUSSIAN_C,
    thresholdType=cv.THRESH_BINARY_INV,
    blockSize=11,
    C=2
)
```

- Phân tích:

+ **import cv2 as cv, import numpy as np**: Nạp thư viện OpenCV và NumPy, là hai thư viện nền tảng cho mọi tác vụ xử lý ảnh trong Python.

+ **img = cv.imread("input.png")**: Đọc ảnh gốc và giữ lại phiên bản màu. Phiên bản này sẽ được dùng làm "khung vẽ" để hiển thị kết quả cuối cùng.

+ **grayImg = cv.imread("input.png", 0)**: Đọc lại ảnh nhưng với tham số 0 (tương đương cv.IMREAD\_GRAYSCALE), giúp chuyển ảnh trực tiếp sang thang độ xám. Mọi thuật toán phân ngưỡng và phát hiện đường viền đều hoạt động trên ảnh đơn kênh, do đó đây là bước bắt buộc.

+ **binary\_img = cv.adaptiveThreshold(...)**: Đây là một lựa chọn kỹ thuật quan trọng và hiệu quả hơn so với phân ngưỡng toàn cục (global thresholding).

⇒ **Lý do lựa chọn**: Thay vì dùng một ngưỡng chung cho toàn ảnh (như Otsu), Phân ngưỡng Thích ứng sẽ tính toán ngưỡng riêng cho từng vùng nhỏ. Điều này đặc biệt hữu ích cho các ảnh có điều kiện chiếu sáng không đồng đều, giúp bảo toàn chi tiết ở cả vùng sáng và vùng tối.

+ **adaptiveMethod=cv.ADAPTIVE\_THRESH\_GAUSSIAN\_C**: Ngưỡng tại một pixel được tính là giá trị trung bình theo trọng số Gaussian của các pixel trong vùng lân cận (blockSize), trừ đi hằng số C. Phương pháp Gaussian cho kết quả mượt mà và tự nhiên hơn so với trung bình cộng thông thường.

+ **thresholdType=cv.THRESH\_BINARY\_INV**: Đảo ngược kết quả, biến ký tự (vốn tối hơn nền) thành màu trắng (255) và nền thành màu đen (0). Đây là định dạng yêu cầu của hàm findContours vì nó tìm các đối tượng màu trắng.<sup>1</sup>

+ **blockSize=11**: Kích thước của vùng lân cận để tính ngưỡng là 11x11 pixel. Giá trị này phải là số lẻ.

+ **C=2**: Một hằng số được trừ đi từ giá trị trung bình, giúp tinh chỉnh và làm sắc nét kết quả phân ngưỡng.

## **Bước 2: Tìm Contours và Xây dựng công cụ hợp nhất**

- Mục tiêu: Tìm các đường viền ban đầu từ ảnh nhị phân và xây dựng một công cụ thông minh để khắc phục tình trạng một ký tự bị nhiễu làm vỡ thành nhiều mảnh.

```
# Tìm contours trong ảnh nhị phân
contours, _ = cv.findContours(binary_img, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)

def merge_boxes_vertically(boxes, v_threshold=10, h_threshold=5):
    """Gộp các bounding boxes gần nhau theo chiều dọc"""
    if not boxes:
        return boxes

    # Sắp xếp theo tọa độ x, sau đó y
    boxes = sorted(boxes, key=lambda b: (b[0], b[1]))
    merged = []

    for x, y, w, h in boxes:
        merged_flag = False

        for i, (mx, my, mw, mh) in enumerate(merged):
            # Kiểm tra overlap ngang và khoảng cách dọc
            h_overlap = not (x + w < mx or mx + mw < x)
            v_distance = abs(y - (my + mh)) if y > my else abs(my - (y + h))

            # Nếu đủ điều kiện thì gộp lại
            if h_overlap and v_distance < v_threshold:
                new_x = min(x, mx)
                new_y = min(y, my)
                new_w = max(x + w, mx + mw) - new_x
                new_h = max(y + h, my + mh) - new_y
                merged[i] = (new_x, new_y, new_w, new_h)
                merged_flag = True
                break

        if not merged_flag:
            merged.append((x, y, w, h))

    return merged
```

- Phân tích:

- + **contours, \_ = cv.findContours(...):** Tìm các đường viền trên ảnh nhị phân.
- + **cv.RETR\_EXTERNAL:** Chỉ lấy các đường viền ngoài cùng, là lựa chọn hiệu quả nhất để phát hiện các ký tự như những đối tượng độc lập, bỏ qua các lỗ bên trong.<sup>4</sup>

- + **cv.CHAIN\_APPROX\_SIMPLE**: Nén các đường viền bằng cách chỉ lưu trữ các điểm cuối của các đoạn thẳng, giúp tiết kiệm bộ nhớ và tăng tốc độ xử lý.<sup>1</sup>
- + **def merge\_boxes\_vertically(...)**: Hàm này là một giải pháp tùy chỉnh, được tạo ra để giải quyết một vấn đề thực tế: nhiều hoặc quá trình xử lý có thể làm một ký tự bị đứt gãy (ví dụ, số 8 bị tách thành hai vòng tròn riêng biệt). Hàm này sẽ "hàn" chúng lại.
- + **boxes = sorted(...)**: Sắp xếp các box theo tọa độ từ trái qua phải, trên xuống dưới để đảm bảo quá trình hợp nhất diễn ra một cách tuần tự và logic.
- + Logic cốt lõi nằm trong vòng lặp lồng nhau, so sánh mỗi box mới với các box đã được hợp nhất. Điều kiện **if h\_overlap and v\_distance < v\_threshold**: là chìa khóa:
  - + **h\_overlap**: Kiểm tra xem hai box có chồng lấn hoặc nằm sát nhau theo chiều ngang không.
  - + **v\_distance < v\_threshold**: Kiểm tra xem khoảng cách theo chiều dọc giữa chúng có đủ nhỏ hay không.
  - + Nếu cả hai điều kiện đều đúng, hai box được coi là thuộc cùng một ký tự và được hợp nhất thành một box lớn hơn bao trọn cả hai.

### Bước 3: Xử lý và Vẽ kết quả từ Giai đoạn tổng thể

- **Mục tiêu**: Lọc các contours không hợp lệ, áp dụng hàm hợp nhất, và vẽ các bounding box từ kết quả xử lý trên toàn bộ ảnh.

```

# Định nghĩa các vùng ROI cần xử lý đặc biệt
ROI_REGIONS = [
    (494, 558, 250, 300),
    (494, 558, 300, 369),
    (395, 474, 280, 437),
    (207, 263, 360, 395),
    (299, 365, 330, 373),
    (302, 368, 425, 472),
    (493, 560, 365, 408),
    (500, 567, 419, 469),
    (300, 378, 468, 500)
]

# Giới hạn diện tích contour
MIN_AREA, MAX_AREA = 150, 1450

# Thu thập và merge các bounding boxes
all_boxes = [cv.boundingRect(cnt) for cnt in contours]
if MIN_AREA < cv.contourArea(cnt) < MAX_AREA]
merged_boxes = merge_boxes_vertically(all_boxes, v_threshold=15)

# Vẽ boxes với expand có điều kiện (chỉ số đầu tiên ở mép trái)
img_h, img_w = img.shape[:2]
first_expanded = False

for x, y, w, h in merged_boxes:
    # Chỉ expand chiều dọc cho số đầu tiên sát mép trái (x < 1)
    if x < 1 and not first_expanded:
        expand = 6
        new_y = max(0, y - expand)
        new_h = min(img_h - new_y, h + 2 * expand)
        cv.rectangle(img, (x, new_y), (x + w, new_y + new_h), (0, 255, 0), 2)
        first_expanded = True
    else:
        cv.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)

```

#### - Phân tích :

- + **ROI\_REGIONS:** Đây là danh sách các tọa độ (y1, y2, x1, x2) được xác định thủ công sau khi quan sát thấy thuật toán tổng thể thất bại ở những vùng này. Chúng sẽ được xử lý riêng ở bước sau.
- + **MIN\_AREA, MAX\_AREA:** Các ngưỡng diện tích được xác định qua thử nghiệm để lọc bỏ các nhiễu nhỏ và các đối tượng không phải ký tự.
- + **all\_boxes = [...]:** Một list comprehension được dùng để duyệt qua tất cả contours, tính diện tích (cv.contourArea), và chỉ giữ lại bounding box (cv.boundingRect) của những contour hợp lệ.<sup>5</sup>

- + **merged\_boxes = merge\_boxes\_vertically(...)**: Áp dụng hàm đã định nghĩa ở trên để hợp nhất các ký tự bị vỡ.
- + Vòng lặp **for x, y, w, h in merged\_boxes**: Duyệt qua các box cuối cùng để vẽ.
- + **if x < 1 and not first\_expanded::** Đây là một xử lý đặc biệt, một tình hình nhỏ nhưng quan trọng. Nó được thêm vào sau khi quan sát thấy ký tự đầu tiên ở một số dòng (sát lề trái,  $x < 1$ ) có thể bị cắt hoặc nhận diện chưa đủ chiều cao. Đoạn code này sẽ "mở rộng" (expand) bounding box theo chiều dọc một chút để bao phủ ký tự tốt hơn. Cờ `first_expanded` đảm bảo việc này chỉ xảy ra một lần cho mỗi dòng.

#### **Bước 4: Định nghĩa chiến lược xử lý chuyên biệt cho ROI**

- **Mục tiêu:** Xây dựng một hàm đa năng, có khả năng tùy biến cao để xử lý các vùng ảnh nhỏ, nơi có các loại nhiễu và vấn đề khác nhau.

```
def detect_and_draw(roi, full_img, offset_y, offset_x, iterations, kernel_size, threshold):
    """Detect và vẽ contours trong ROI"""
    # Chuẩn bị kernel và xử lý ảnh
    kernel = np.ones((kernel_size, kernel_size), np.uint8)
    closed = cv.morphologyEx(roi, cv.MORPH_CLOSE, kernel)
    gray = cv.cvtColor(closed, cv.COLOR_BGR2GRAY)
    _, binary = cv.threshold(gray, threshold, 255, cv.THRESH_BINARY)
    eroded = cv.erode(binary, kernel, iterations=iterations)

    # Tìm contours trong ROI
    contours, _ = cv.findContours(eroded, cv.RETR_CCOMP, cv.CHAIN_APPROX_SIMPLE)

    # Thu thập boxes trong ROI
    boxes = [cv.boundingRect(cnt) for cnt in contours]
    if 300 < cv.contourArea(cnt) < 1600]
    merged = merge_boxes_vertically(boxes, v_threshold=15, h_threshold=3)

    # Vẽ boxes với expand có điều kiện
    roi_h, roi_w = roi.shape[:2]
    img_h, img_w = full_img.shape[:2]
    first_expanded = False

    for x, y, w, h in merged:
        # Chỉ expand chiều dọc cho số đầu tiên ở mép trái ROI
        if x < 1 and not first_expanded:
            expand = 6
            new_y = max(0, y - expand)
            new_h = min(roi_h - new_y, h + 2 * expand)
            final_coords = (x + offset_x, new_y + offset_y,
                           min(img_w, x + offset_x + w),
                           min(img_h, new_y + offset_y + new_h))
            first_expanded = True
        else:
            final_coords = (x + offset_x, y + offset_y,
                           min(img_w, x + offset_x + w),
                           min(img_h, y + offset_y + h))

        cv.rectangle(full_img, final_coords[:2], final_coords[2:], (0, 255, 0), 2)
```

#### - Phân tích:

- + **def detect\_and\_draw(...):** Hàm này là trái tim của giai đoạn xử lý chuyên biệt, nhận vào ROI và một loạt tham số tùy chỉnh.
- + **kernel = np.ones(...):** Tạo kernel hình vuông với kích thước kernel\_size được truyền vào, cho phép điều chỉnh mức độ ảnh hưởng của các phép toán hình thái.
- + **closed = cv.morphologyEx(roi, cv.MORPH\_CLOSE, kernel):** Áp dụng Phép Đóng (Closing), là phép giãn nở rồi đến phép co. Phép này rất hiệu quả



trong việc lấp đầy các lỗ nhỏ hoặc các vết nứt bên trong ký tự, giúp chúng trở nên liền mạch hơn trước khi phân ngưỡng.<sup>6</sup>

+ **\_ , binary = cv.threshold(...)**: Sử dụng phân ngưỡng cố định với giá trị threshold được truyền vào. Đối với một vùng ảnh nhỏ và đã được tiền xử lý, ngưỡng cố định thường cho kết quả ổn định và dễ kiểm soát hơn.

+ **eroded = cv.erode(binary, kernel, iterations=iterations)**: Áp dụng thêm Phép Co (Erosion) để làm mỏng ký tự hoặc loại bỏ các cầu nối nhiều yếu. Tham số iterations cho phép điều chỉnh mức độ co mạnh hay nhẹ tùy theo đặc điểm của ROI.<sup>7</sup>

+ **cv.findContours(eroded, cv.RETR\_CCOMP,...)**: Sử dụng RETR\_CCOMP để truy xuất các đường viền theo cấu trúc 2 cấp (viên ngoài và viên lỗ bên trong). Lựa chọn này phù hợp cho các ký tự phức tạp hoặc khi nhiều tạo ra các "lỗ" giả bên trong ký tự, giúp việc lọc sau này có thể linh hoạt hơn.<sup>4</sup>

+ Phần còn lại của hàm thực hiện lọc diện tích, hợp nhất và vẽ. Điểm quan trọng là final\_coords được tính toán bằng cách cộng tọa độ cục bộ (x, y) với tọa độ offset của ROI, đảm bảo hình chữ nhật được vẽ đúng vị trí trên ảnh gốc full\_img.

### **Bước 5: Thực thi xử lý chuyên biệt và Hoàn thiện**

- **Mục tiêu:** Áp dụng hàm detect\_and\_draw cho từng ROI đã định nghĩa với bộ tham số tương ứng và lưu ảnh cuối cùng.



```

# Xử lý từng ROI với các tham số tương ứng
ROI_PARAMS = {
    (494, 250): (1, 2, 1),
    (494, 300): (1, 3, 1),
    (395, 280): (1, 3, 30),
    (207, 360): (2, 1, 1),
    (299, 330): (1, 2, 10),
    (302, 425): (3, 2, 10),
    (493, 365): (2, 3, 1),
    (500, 419): (2, 2, 1),
    (300, 468): (2, 3, 10)
}

for y1, y2, x1, x2 in ROI_REGIONS:
    roi = img[y1:y2, x1:x2].copy()
    params = ROI_PARAMS.get((y1, x1))
    if params:
        detect_and_draw(roi, img, y1, x1, *params)

# Lưu ảnh kết quả
cv.imwrite("output.png", img)

```

#### - Phân tích:

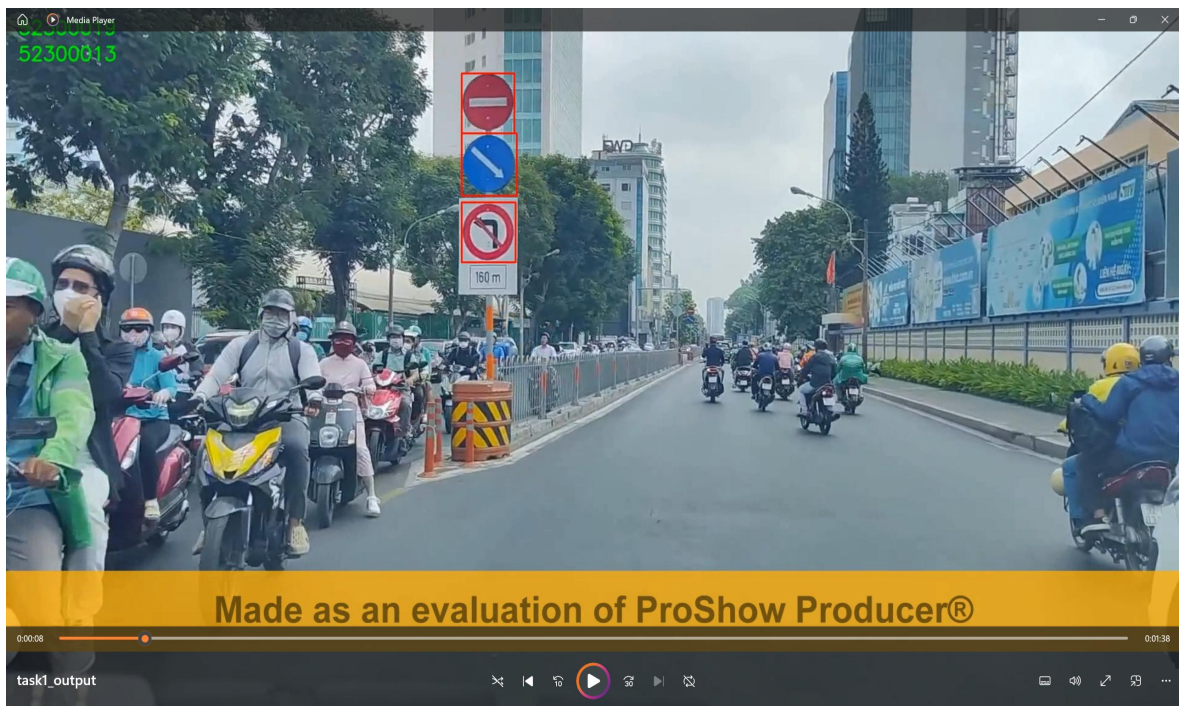
- + **ROI\_PARAMS**: Một dictionary lưu trữ các bộ tham số tùy chỉnh. key là tọa độ góc trên bên trái của ROI, value là một tuple chứa (iterations, kernel\_size, threshold). Cấu trúc này cho phép liên kết mỗi ROI với một chiến lược xử lý riêng, thể hiện sự tinh chỉnh ở mức độ cao.
- + Vòng lặp **for y1, y2, x1, x2 in ROI\_REGIONS**: Duyệt qua danh sách các vùng cần xử lý đặc biệt.
- + **roi = img[y1:y2, x1:x2].copy()**: Cắt ra vùng ảnh ROI từ ảnh gốc.
- + **params = ROI\_PARAMS.get((y1, x1))**: Lấy ra bộ tham số tương ứng với ROI này từ dictionary.
- + **if params**:: Kiểm tra xem ROI này có bộ tham số được định nghĩa hay không.
- + **detect\_and\_draw(roi, img, y1, x1, \*params)**: Gọi hàm xử lý chuyên biệt. Toán tử \* sẽ giải nén tuple params thành các đối số riêng lẻ (iterations, kernel\_size, threshold) cho hàm.

+ `cv.imwrite("output.png", img)`: Sau khi cả hai giai đoạn (tổng thể và chuyên biệt) đã vẽ xong kết quả lên `img`, hàm này sẽ lưu lại ảnh cuối cùng, hoàn thành bài toán.

## CHƯƠNG 2. KẾT QUẢ CÁC BÀI TOÁN

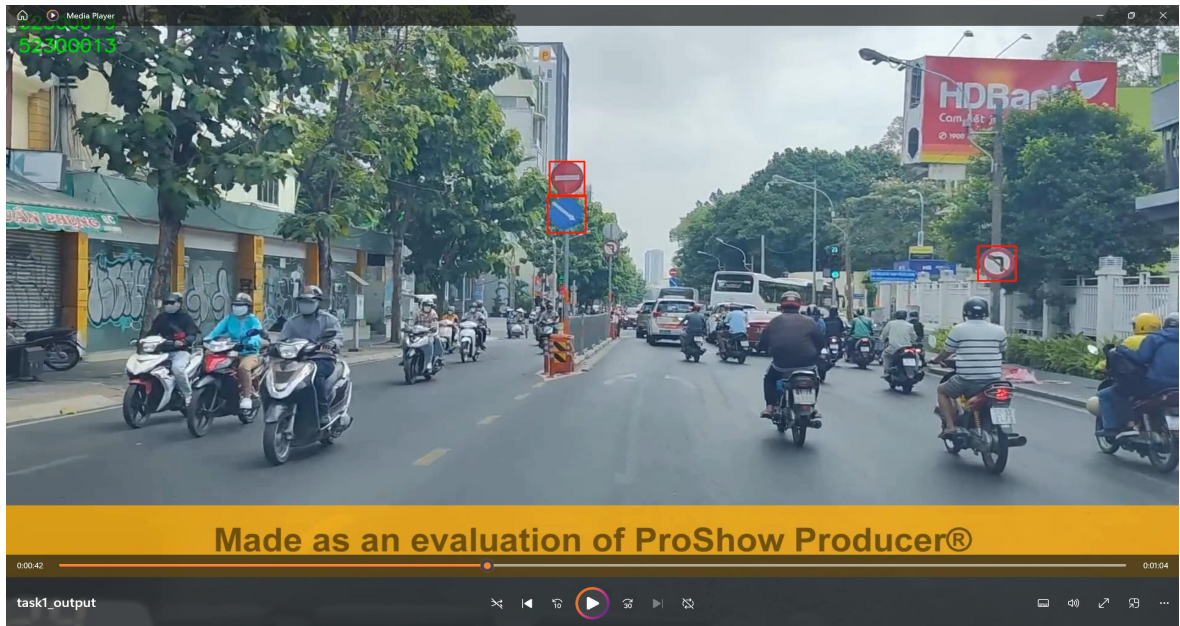
### 2.1 Bài toán 1

#### 2.1.1 Output 1



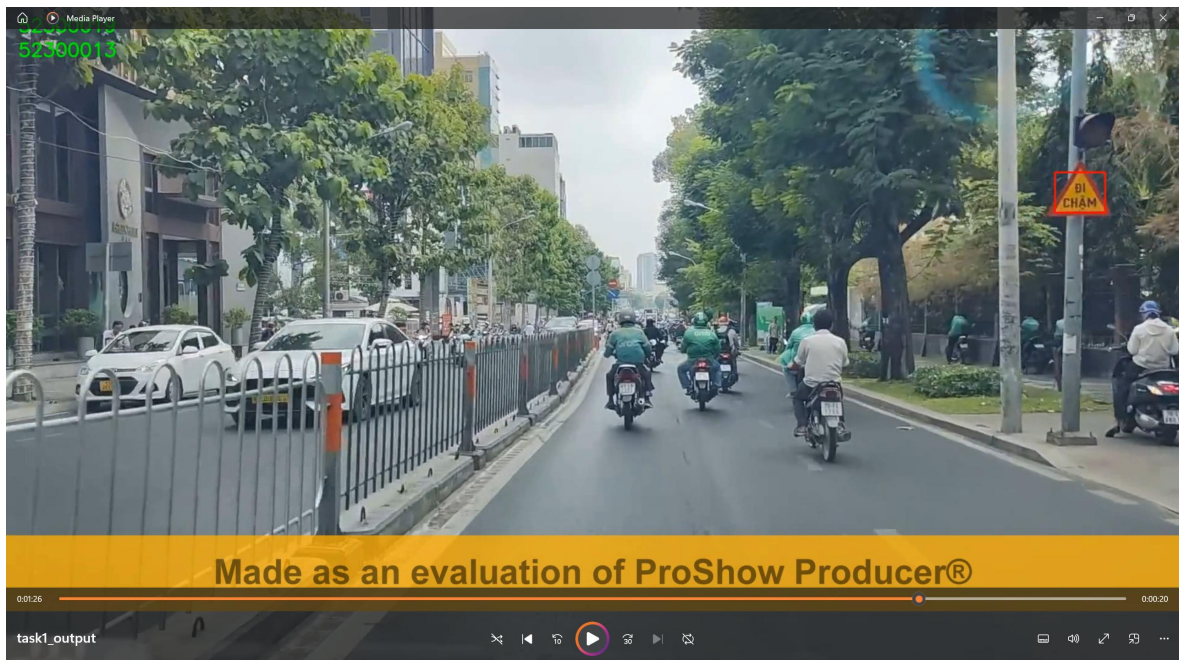
Hình 1: Output 1

### 2.1.2 Output 2



Hình 2: Output 2

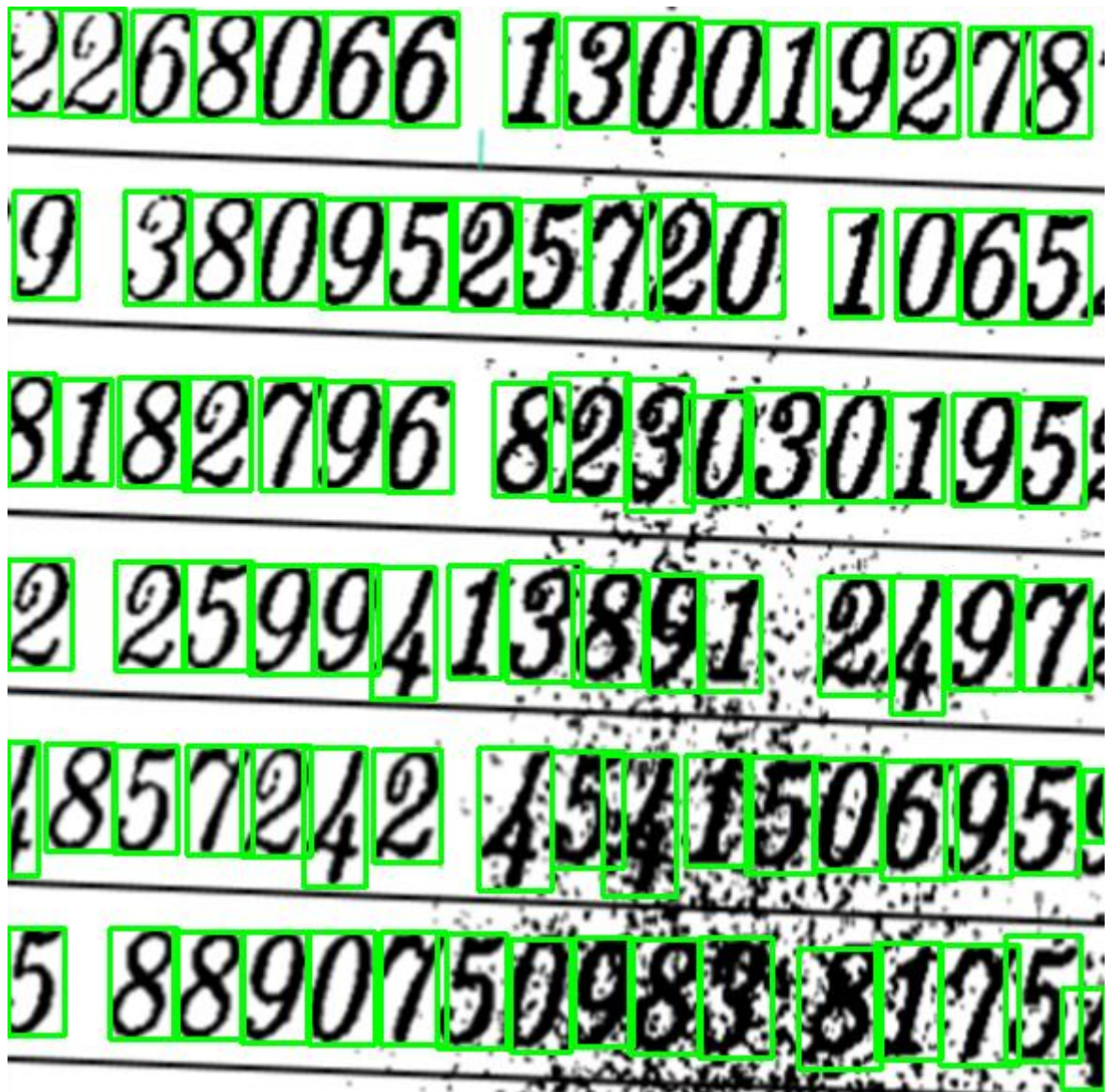
### 2.1.3 Output 3



Hình 3: Output 3



## 2.2 Bài toán 2



Hình 4 Kết quả output.png bài toán 2

## CHƯƠNG 3. KẾT LUẬN

### 3.1 Kết luận

Bài báo cáo đã trình bày các giải pháp toàn diện sử dụng thư viện OpenCV và ngôn ngữ Python để giải quyết thành công hai bài toán xử lý ảnh và video.

Đối với bài toán 1, hệ thống đã phát hiện chính xác các biển báo giao thông trong video bằng cách phân tích không gian màu HSV và áp dụng các phép biến đổi hình thái học. Đối với bài toán 2, một chiến lược xử lý "lai" kết hợp giữa phương pháp tổng thể và chuyên biệt theo từng vùng đã được triển khai hiệu quả, giúp nhận diện thành công các ký tự số ngay cả trong điều kiện nhiễu nặng. Cả hai giải pháp đều đáp ứng đầy đủ các yêu cầu của đề bài, chứng tỏ khả năng vận dụng vững chắc các kiến thức xử lý ảnh vào thực tiễn.

### 3.2 Hướng phát triển

Dù đã hoàn thành mục tiêu, các giải pháp này vẫn có thể được cải tiến và mở rộng. Đối với bài toán 1, có thể tích hợp các mô hình học máy như Mạng Nơ-ron Tích chập (CNN) để không chỉ phát hiện mà còn phân loại được từng loại biển báo cụ thể, tăng cường tính thông minh cho hệ thống. Đối với bài toán 2, hướng phát triển tự nhiên là xây dựng một hệ thống Nhận dạng Ký tự Quang học (OCR) hoàn chỉnh, có khả năng tự động nhận dạng giá trị của các con số đã được khoanh vùng, mở ra nhiều ứng dụng thực tế như đọc đồng hồ đo hay số hóa tài liệu.

## TÀI LIỆU THAM KHẢO

### 3.3 Tiếng Việt

Ngô Diên Tập (2001). Lý thuyết xử lý ảnh. Nhà xuất bản Khoa học và Kỹ thuật.

Univtechnews (2017). [Tìm hiểu thuật toán Otsu](#).

Viblo (2019). [Ứng dụng xử lý ảnh trong thực tế với thư viện OpenCV](#)  
[Xử lý ảnh với OpenCV - Xác định viền trong ảnh \(Edge Detection\)](#).

### 3.4 Tiếng Anh

Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson.

Suzuki, S., & Abe, K. (1985). Topological Structural Analysis of Digitized Binary Images by Border Following. Computer Vision, Graphics, and Image Processing, 30(1), 32-46.

Bradley, D., & Roth, G. (2007). Adaptive Thresholding using the Integral Image. Journal of Graphics Tools, 12(2), 13-21.

OpenCV Documentation. (n.d.). [Contour Features](#).

OpenCV Documentation. (n.d.). [Image Thresholding](#).